

Basic Blocks of a Computer

A computer system, at its core, is built around a few fundamental building blocks that work together to process information. Here's a breakdown of these basic blocks:

1. **Central Processing Unit (CPU):** The "brain" of the computer. It's responsible for executing instructions and performing calculations. The CPU itself contains several key components:
 - **Arithmetic Logic Unit (ALU):** Performs arithmetic operations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT).
 - **Control Unit (CU):** Directs the operations of the ALU, memory, and I/O devices. It fetches instructions, decodes them, and generates the necessary control signals to execute them.
 - **Registers:** Small, high-speed storage locations within the CPU used to hold data and instructions that are actively being processed.
2. **Memory (Main Memory or RAM):** Stores data and instructions that the CPU is currently using or needs to access quickly. It's volatile, meaning its contents are lost when the power is turned off. Think of it as the computer's short-term memory.
3. **Input/Output (I/O) Devices:** Allow the computer to interact with the outside world.
 - **Input Devices:** Provide data to the computer (e.g., keyboard, mouse, scanner, microphone).
 - **Output Devices:** Display or present processed information from the computer (e.g., monitor, printer, speakers).
4. **System Bus:** A set of electrical pathways that connect the CPU, memory, and I/O devices, enabling them to communicate with each other. It typically consists of three types of buses:
 - **Data Bus:** Carries the actual data being transferred.
 - **Address Bus:** Carries the memory addresses of the data.
 - **Control Bus:** Carries control signals that coordinate the activities of the different components.

How These Blocks Interact

Here's a simplified view of how these basic blocks work together:

1. The CPU fetches instructions from memory.
2. The CU decodes the instruction.
3. If the instruction involves data, the CPU may read the data from memory or from an input device.
4. The ALU performs the operation on the data.

5. The result is stored in memory or sent to an output device.

This cycle repeats continuously, allowing the computer to execute programs and process information.

In essence:

- The CPU is the active processing element.
- Memory holds the data and instructions.
- I/O devices provide the interface to the external world.
- The system bus enables communication between all these components.

These four basic blocks form the foundation of virtually all computer systems, from tiny embedded devices to powerful supercomputers. While specific implementations can be incredibly complex, these core components and their interactions remain fundamental.

Register Transfer Language

Register Transfer Language (RTL) is a notation used to describe the operations and data flow within a digital system, particularly at the level of registers and the logical operations performed on them. It's a crucial abstraction in digital design, bridging the gap between high-level descriptions of a system's behavior and the actual hardware implementation.

Think of it as a precise, step-by-step description of how data moves and is transformed within a digital circuit, focusing on the registers where the data is stored and the operations that manipulate it.

Key Concepts in RTL

1. **Registers:** These are the fundamental storage elements in a digital system. They hold data, instructions, or control signals. In RTL, registers are represented by names (e.g., R1, PC, MAR).
2. **Signals:** Signals represent the flow of data or control information within the system. They can be thought of as wires carrying binary values.
3. **Data Transfer:** RTL describes how data is transferred between registers. This is typically done using an assignment operator (e.g., <- or =). For example:
 4. R1 <- R2; // Data from register R2 is transferred to register R1
5. **Operations:** RTL includes operators to represent logical and arithmetic operations performed on data. Examples include:
 - Arithmetic: +, -, *, /
 - Logical: AND, OR, NOT, XOR

- Shift/Rotate: <<, >>
- 6. **Control Signals:** Control signals govern when data transfers and operations occur. They can be used to enable or disable specific operations based on conditions.
- 7. **Clock Cycles:** RTL descriptions are often timed, meaning operations are associated with specific clock cycles. This reflects the synchronous nature of digital circuits.

Example RTL Description

Let's say we want to describe the operation of adding two registers, R1 and R2, and storing the result in R3. In RTL, this might look like:

Clock Cycle 1:

```
R3 <- R1 + R2; // Add the contents of R1 and R2, store in R3
```

Importance of RTL

- **Precise Description:** RTL provides a precise and unambiguous way to describe the behavior of digital systems.
- **Hardware Synthesis:** RTL descriptions can be automatically translated into hardware implementations using synthesis tools. This is a key part of the digital design process.
- **Simulation and Verification:** RTL models can be simulated to verify their correctness before they are implemented in hardware.
- **Design and Documentation:** RTL serves as a valuable design and documentation tool for complex digital systems.

Levels of Abstraction

RTL is a relatively low-level description, focusing on the register-level operations. There are higher levels of abstraction, such as algorithmic or behavioral descriptions, that can be used to describe a system at a more abstract level. However, RTL is essential for bridging the gap to the actual hardware implementation.

Hardware Description Languages (HDLs)

RTL is typically expressed using Hardware Description Languages (HDLs) like Verilog and VHDL. These languages provide the syntax and semantics for describing digital systems at the RTL level. Synthesis tools then take this HDL code and convert it into a gate-level netlist, which can be implemented in hardware.

In Summary

RTL is a fundamental tool in digital design. It provides a way to precisely describe the data flow and operations within a digital system, enabling hardware synthesis, simulation, and verification. It's the language used to specify how the registers in a digital circuit interact and what operations are performed on the data they hold.

Functions of a Control Unit

The Control Unit (CU) is the "brain" within the CPU, responsible for orchestrating all operations within the computer. It's like the conductor of an orchestra, ensuring all the different parts play together harmoniously. Here's a breakdown of its key functions:

1. **Instruction Fetching:** The CU retrieves instructions from memory. It fetches the next instruction to be executed from the memory location pointed to by the Program Counter (PC).
2. **Instruction Decoding:** Once an instruction is fetched, the CU decodes it to understand what operation needs to be performed. It identifies the opcode (the operation to be carried out) and the operands (the data or memory locations involved).
3. **Control Signal Generation:** Based on the decoded instruction, the CU generates control signals. These signals are like commands that direct other components of the CPU (ALU, registers) and the memory to perform specific actions. For example, control signals might tell the ALU which operation to perform (addition, subtraction, etc.), which registers to use as input, and where to store the result.
4. **Data Transfer Management:** The CU manages the flow of data within the CPU and between the CPU and memory. It controls when data is read from or written to memory, and it directs the movement of data between registers.
5. **Timing and Synchronization:** The CU ensures that all operations are performed in the correct sequence and at the right time. It uses clock signals to synchronize the activities of the different components, ensuring that data is available when needed and that operations are completed before the next instruction is fetched.
6. **Interrupt Handling:** The CU handles interrupts, which are signals from external devices or software that require the CPU's attention. When an interrupt occurs, the CU saves the current state of the CPU, handles the interrupt, and then returns to the original program.
7. **Program Flow Control:** The CU manages the execution flow of a program, including jumps, branches, and subroutine calls. It updates the Program Counter (PC) to point to the next instruction to be executed, ensuring that instructions are executed in the correct order.

In simpler terms:

The Control Unit acts like a manager:

- It fetches instructions (like getting tasks from a to-do list).
- It decodes instructions (understanding what needs to be done).
- It issues commands (control signals) to other parts of the CPU and memory to carry out the tasks.
- It manages the flow of information (data) between different parts.

- It keeps everything synchronized and running smoothly.
- It handles interruptions (urgent requests that need immediate attention).
- It controls the order in which tasks are done (program flow).

The Control Unit is absolutely essential for the operation of a computer. Without it, the CPU would be unable to execute instructions or perform any meaningful work. It's the conductor that makes all the other components work together to process information.

SRAM with Diagram

SRAM stands for **Static Random Access Memory**. It is a form of a semiconductor. It is widely used in [microprocessors](#), general computing applications, and electronic devices. The SRAM is volatile which means the data stored in it gets all wiped out once the power supply is cut. SRAM comprised of flip flops. It consists of 4-6 transistors, once the flip flop stores the bit it keeps it stored until the opposite bit is stored in it.

History of SRAM

Engineer John Schmidt invented the SRAM in 1964 at Fairchild Semiconductors. The first SRAM is 64-bit and uses p-channel MOS.

Intel released its first 256-bit Intel 1101 SRAM chip in 1969, five years after its invention. But it uses Schottky TTL (Transistor-Transistor-Logic) architecture for its build.

Early SRAMs were manufactured using ceramic plastic. But nowadays, SRAMs are integrated directly to the CPU for faster and better processing.

Characteristics

- **The data is held statically:** The data is stored statically in SRAM and it doesn't need to be refreshed unlike DRAMs.
- **It is a type of Random Access Memory:** The SRAM is a type of Random Access Memory. Random Access Memories are those from which the data can be accessed (read/write) randomly (means any memory location can be accessed), regardless of the memory location that was accessed earlier.
- **It uses flip flop for storing data:** It uses flip flops to store bits. Each flip flop is made up of 4-6 transistors.
- **It is used as a Cache Memory in CPU:** SRAM is used as [cache memory](#) for CPUs as they are faster and stores data statically.

Advantages

- It is faster to access and perform operations like read & write.
- The data can be accessed randomly.

- It is used as a cache memory.
- It doesn't need to be refreshed as it stored data statically.
- It has medium power consumption. It requires less power as compared to DRAM.

Disadvantages

- It is expensive.
- It is volatile in nature i.e., data is lost when the memory is not powered.
- It has a low storage capacity.
- It is not possible to refresh the program.
- It has a more complex design and they are bigger in size as well when compared to [DRAM](#).
- It reduces the memory density.

Examples of SRAM

1. Cache memory:

- L1, L2, and L3 cache in CPUs
- Cache in [GPUs](#)

2. Register files in microprocessors

3. Small amounts of memory in microcontrollers

Limitations in instruction pipeline

Instruction pipelining, while a powerful technique for boosting CPU performance, isn't without its limitations. Here are some of the key challenges and limitations associated with pipelining:

1. **Pipeline Hazards:** These are situations that prevent the next instruction in the pipeline from being executed in its designated clock cycle. They disrupt the smooth flow of instructions and can lead to stalls or incorrect results. The three main types are:
 - **Structural Hazards:** Occur when multiple instructions try to access the same hardware resource (e.g., memory, ALU) at the same time.
 - **Data Hazards:** Arise when an instruction depends on the result of a previous instruction that is still in the pipeline (RAW, WAR, WAW dependencies).
 - **Control Hazards:** Caused by branch instructions, which can change the program's flow, making it difficult to fetch the correct next instruction.
2. **Increased Hardware Complexity:** Implementing pipelining requires additional hardware, such as registers between pipeline stages, hazard detection and resolution logic, and potentially more complex control units. This adds to the cost and complexity of the CPU.

3. **Instruction Latency:** While pipelining increases throughput (the number of instructions completed per unit time), it doesn't necessarily reduce the latency of a *single* instruction. In fact, the latency of an individual instruction might be slightly higher due to the overhead of pipelining.
4. **Uneven Pipeline Stages:** If the different stages of the pipeline take significantly different amounts of time, it can create bottlenecks. The slowest stage will determine the overall pipeline clock speed, limiting the performance gains.
5. **Branch Penalties:** Control hazards introduced by branch instructions can significantly impact pipeline performance. Branch prediction techniques can mitigate this, but mispredictions lead to flushing the pipeline and restarting with the correct instructions, which is costly.
6. **Increased Power Consumption:** The additional hardware and increased clock speeds associated with pipelining can lead to higher power consumption.
7. **Debugging Complexity:** Pipelining can make debugging more challenging because instructions are executed concurrently and their effects can be intertwined. Tracing the execution flow and identifying the source of errors can be more complex.
8. **Compiler Dependence:** The effectiveness of pipelining can depend on the compiler's ability to generate code that minimizes hazards. Compilers play a role in scheduling instructions to avoid data hazards and in branch prediction strategies.
9. **Limited Scalability:** While increasing the number of pipeline stages can initially improve performance, there are diminishing returns. Eventually, the overhead of managing the pipeline and the increased impact of hazards can outweigh the benefits.
10. **Interrupt Handling Complexity:** Handling interrupts in a pipelined processor can be more complex because the pipeline might need to be flushed or stalled to ensure correct interrupt handling.

In summary, while pipelining is a crucial technique for achieving high performance, it introduces a range of challenges related to hardware complexity, hazard management, and overall system design. Careful consideration of these limitations is essential for designing efficient and reliable pipelined processors.

Difference between Program and Process

- **Program:** A program is like a recipe. It's a set of instructions, written in a programming language, that describes how to perform a task.¹ It's a static entity, meaning it doesn't do anything on its own. It's like a file stored on your hard drive, waiting to be used.
- **Process:** A process is like a chef in a kitchen, actively cooking a dish based on the recipe (the program). It's a dynamic entity, meaning it's a program in execution.² It's what happens when you run a program.

Here's a table summarizing the key differences:

Feature	Program	Process
Nature	Static, passive	Dynamic, active
Location	Stored on disk (secondary memory)	Loaded into RAM (main memory)
Resource Usage	None (until executed)	Requires resources (CPU time, memory, I/O)
Lifespan	Long-term, persists until deleted	Short-term, exists only while running
Execution	Not executable by itself	Actively executing instructions
Instances	One program can have multiple processes running	Each process is an instance of a program

Analogy:

Imagine you have a cookbook (program) with a recipe for chocolate cake. When you decide to bake the cake, you gather the ingredients, follow the instructions in the recipe, and bake the cake. The act of baking the cake, with all the ingredients and actions involved, is the process.

Key Takeaways:

- A program is a set of instructions; a process is the execution of those instructions.
- A program is like a blueprint; a process is like the building constructed from that blueprint.
- A program is passive; a process is active.³

Why this distinction matters:

Understanding the difference between programs and processes is crucial for:

- **Multitasking:** Operating systems manage multiple processes concurrently, allowing you to run several programs at the same time.⁴
- **Resource Management:** Operating systems allocate resources (CPU time, memory) to processes, ensuring efficient use of system resources.⁵

- **Debugging:** When debugging, you're often dealing with processes that are misbehaving, not the programs themselves.
-

Part – C

Addressing Modes and its types

Addressing modes are techniques for specifying how the processor accesses operands (data) during instruction execution. They determine how the effective or actual address of an operand is calculated.

Here are the main types of addressing modes:

1. Immediate Addressing Mode

- The operand is included directly in the instruction
- No memory access is needed, making it the fastest mode
- Example: `MOV R1, #25` (loads value 25 directly into register R1)

2. Register Addressing Mode

- The operand is stored in a CPU register
- Very fast since no memory access is required
- Example: `ADD R1, R2` (adds contents of R2 to R1)

3. Direct (Absolute) Addressing Mode

- The instruction contains the actual memory address of the operand
- Single memory access required
- Example: `MOV R1, [200]` (loads value from memory location 200 into R1)

4. Register Indirect Addressing Mode

- A register contains the memory address of the operand
- Useful for accessing arrays and data structures

- Example: MOV R1, [R2] (loads value from address stored in R2 into R1)

5. Indexed Addressing Mode

- Final address = Base address + Index value
- Good for array operations
- Example: MOV R1, [R2 + R3] (base in R2, index in R3)

6. Base-Displacement Addressing Mode

- Final address = Base register + Constant offset
- Useful for accessing structure members
- Example: MOV R1, [R2 + 4] (adds 4 to base address in R2)

7. Auto-Increment/Decrement Mode

- Register value is automatically increased/decreased after or before operand access
- Efficient for sequential data access
- Example: MOV R1, [R2+] (loads value and then increments R2)

8. Relative Addressing Mode

- Address is calculated relative to Program Counter (PC)
- Commonly used in branch instructions
- Example: BRA LABEL (branches to address = PC + offset)

Each addressing mode has its advantages and use cases:

- Immediate mode is best for constant values
- Register mode for frequently accessed data
- Indirect modes for working with pointers and arrays
- Indexed modes for array processing
- Auto-increment/decrement for sequential data processing

The choice of addressing mode affects:

- Instruction execution time
- Program size and memory requirements
- Code flexibility and complexity
- Overall system performance

Modern processors typically support multiple addressing modes to provide flexibility in handling different types of data and programming constructs efficiently.

Restoring techniques in detail

Integer

2)

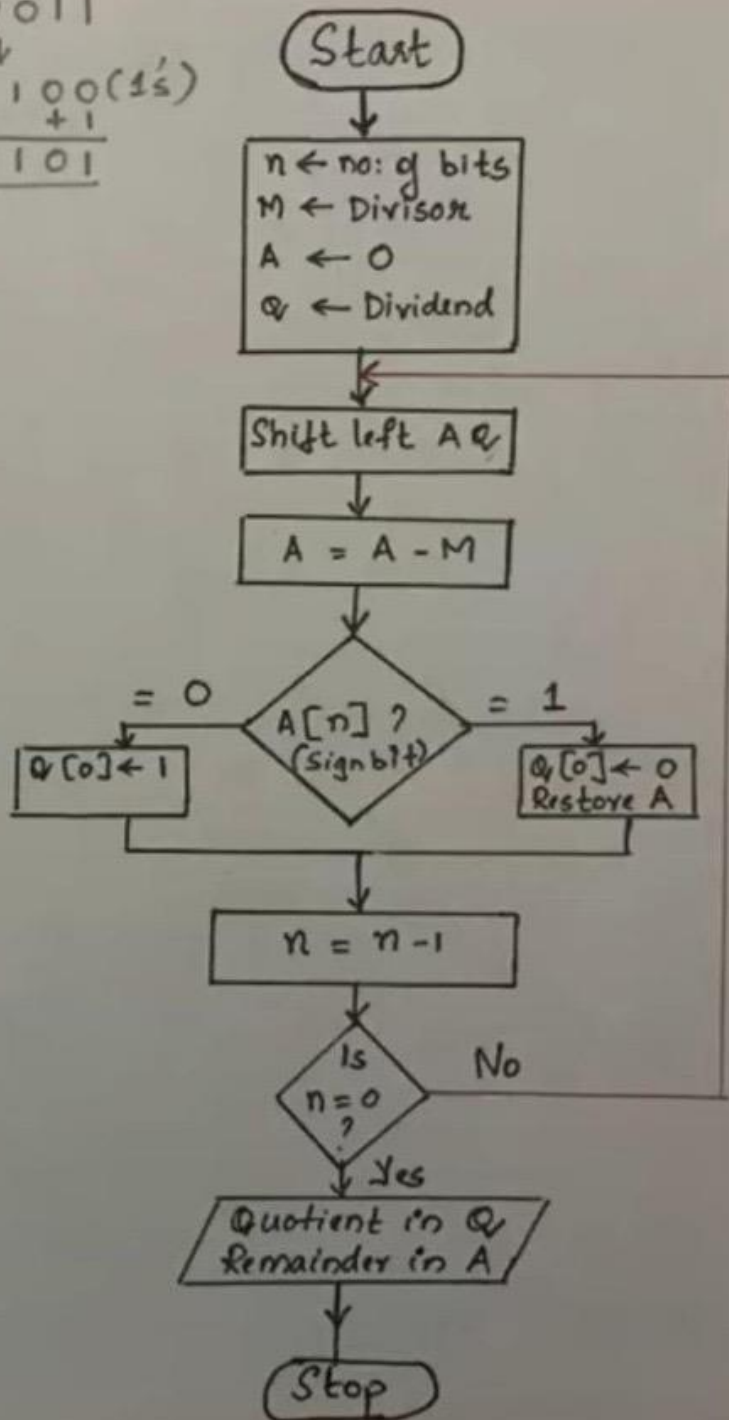
0011

↓

1100 (1's)

+1

1101



Integer

2)

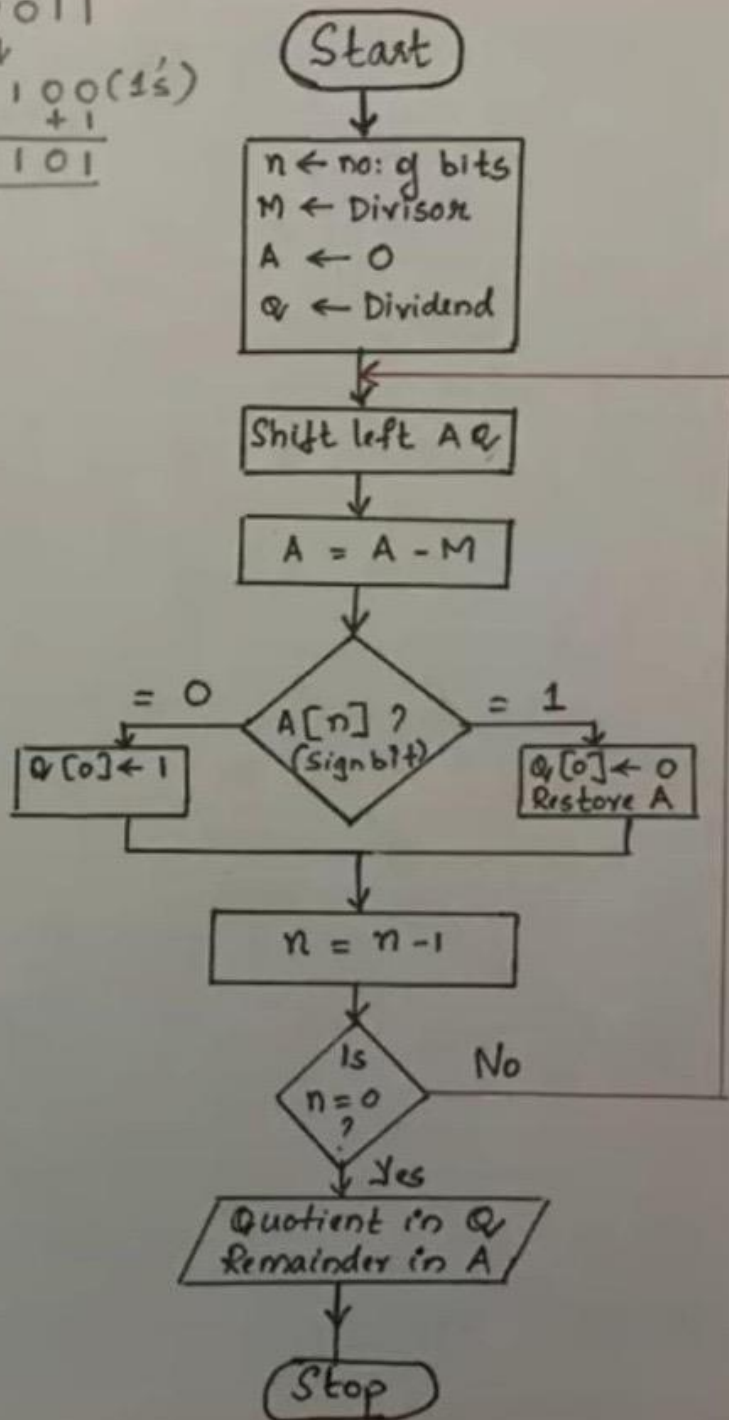
0011

↓

1100 (1's)

+1

1101



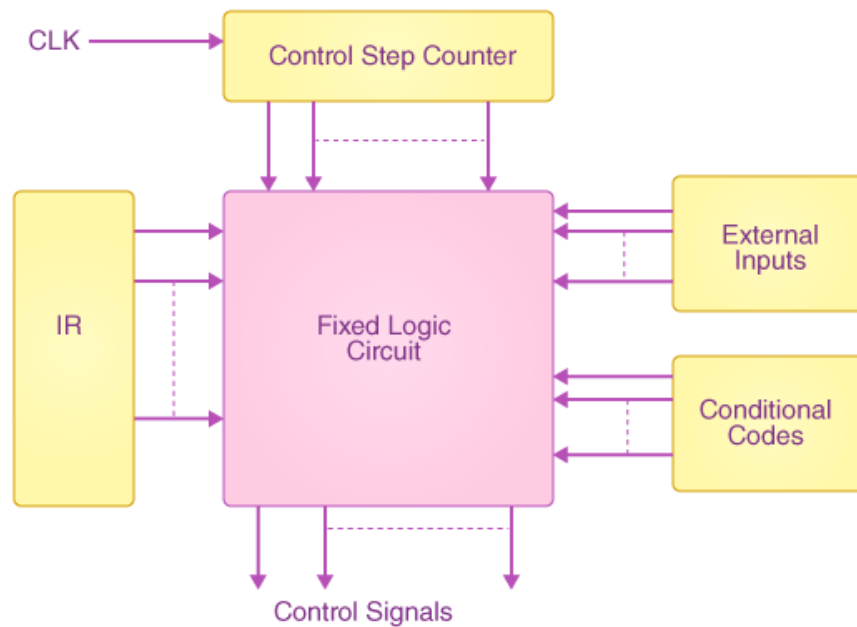
Restoring Division – Example (7/3)

 $M = 0011, \overline{M} + 1 = 1101$

Comment	A	Q	SC
	0000	0111	4
Shl	0000	1110	
$A = A - M$	1101		
	1101		
$A = A + M$	0011		
	0000	1110	3
Shl	0001	1100	
$A = A - M$	1101		
	1110		
$A = A + M$	0011		
	0001	1100	2

Comment	A	Q	SC
Shl	0011	1000	
$A = A - M$	1101		
	0000	1001	1
Shl	0001	0010	
$A = A - M$	1101		
	1110		
$A = A + M$	0011		
	0001	0010	0

Hardwired Control with diagram



Hardwired Control: A control unit implementation where control signals are generated by fixed logic circuits.

Key Characteristics:

- **Speed:** Fast control signal generation due to direct hardware implementation.
- **Implementation:** Complex circuitry, especially for complex instruction sets. Difficult to design and debug.
- **Flexibility:** Inflexible. Any change in instruction set requires hardware redesign.
- **Cost:** Can be expensive due to complex circuitry, especially for CISC architectures.
- **Microinstructions:** Not used. Control signals are generated directly by combinational logic.
- **Applications:** Simpler processors, RISC architectures, where speed is critical.

How it Works:

1. Instruction fetched and decoded.
2. Combinational logic (gates, flip-flops) directly generates the necessary control signals based on the decoded instruction.
3. Control signals activate the appropriate functional units (ALU, memory, registers) to execute the instruction.
4. Process repeats for the next instruction.

Advantages:

- High speed, good for performance-critical applications.

Disadvantages:

- Complex design, difficult to implement and debug.
- Inflexible, hard to modify or extend the instruction set.
- Expensive for complex instruction sets.

Internal Organization of RAM. Discuss structure and Types of RAM

Internal Organization of RAM

RAM (Random Access Memory) is organized internally to allow for efficient storage and retrieval of data. Here's a breakdown of its structure and types:

Structure

1. **Memory Cells:** The fundamental unit of RAM is the memory cell, which can store a single bit of data (0 or 1).¹ These cells are typically implemented using transistors and capacitors.

2. **Array Organization:** Memory cells are arranged in a grid-like structure, forming rows and columns.² Each row represents a memory word, and the intersection of a row and column identifies a specific memory location.
3. **Addressing:** Each memory location is assigned a unique address, which is used to access the data stored in that location.³ The address is provided to the RAM through the address bus.
4. **Data Lines:** Data is transferred to and from the RAM through data lines. The number of data lines determines the width of the memory word (e.g., 8 bits for a byte, 16 bits for a word).
5. **Control Signals:** Control signals, such as Read Enable (RE) and Write Enable (WE), are used to control the read and write operations.⁴

Types of RAM

RAM can be broadly classified into two main types:⁵

1. **Static RAM (SRAM):**
 - **Storage Element:** Uses transistors (typically 6 transistors per bit) to store data.⁶
 - **Speed:** Faster than DRAM.⁷
 - **Complexity:** More complex circuitry, higher cost per bit.⁸
 - **Applications:** Cache memory in CPUs due to its speed.⁹
2. **Dynamic RAM (DRAM):**
 - **Storage Element:** Uses capacitors to store data.¹⁰
 - **Speed:** Slower than SRAM.¹¹
 - **Complexity:** Simpler circuitry, lower cost per bit.¹²
 - **Applications:** Main memory in computers due to its higher density and lower cost.¹³

Further Subcategories of DRAM

- **Synchronous DRAM (SDRAM):** Operates synchronously with the system clock, allowing for faster data transfers.¹⁴
- **Double Data Rate (DDR) SDRAM:** Transfers data on both the rising and falling edges of the clock signal, further increasing the data transfer rate.¹⁵ (DDR2, DDR3, DDR4, DDR5 are evolutions of this technology)¹⁶

Key Considerations

- **Volatility:** RAM is volatile, meaning data is lost when power is turned off.¹⁷
- **Capacity:** RAM capacity is measured in bytes (or kilobytes, megabytes, gigabytes, etc.).¹⁸
- **Access Time:** The time it takes to access data in RAM.

- **Cost:** The cost per bit of storage.

Understanding the internal organization and types of RAM is essential for comprehending how computer memory works and how it impacts system performance.

Different I/O Transfer Methods Used

Here's the detailed answer along with a comparison table for better clarity.

I/O Transfer Methods

I/O (Input/Output) transfer methods determine how data is transferred between a computer's CPU, memory, and peripheral devices. The primary I/O transfer methods are:

1. Programmed I/O (Polling)

- The CPU continuously checks (polls) the status of an I/O device.
- Once the device is ready, the CPU manually transfers data.
- **Pros:** Simple implementation.
- **Cons:** CPU remains busy, leading to inefficiency.
- **Example:** Keyboard input, where the CPU waits for a keypress.

2. Interrupt-Driven I/O

- The CPU initiates a data transfer and performs other tasks.
- When the device is ready, it sends an interrupt signal to the CPU.
- The CPU pauses, handles the request, then resumes its work.
- **Pros:** Reduces CPU idle time.
- **Cons:** Interrupt handling introduces some overhead.
- **Example:** Disk read/write operations.

3. Direct Memory Access (DMA)

- A DMA controller manages data transfer between memory and the I/O device.
- The CPU initiates the transfer and is free for other tasks.
- **Pros:** Faster and more efficient.
- **Cons:** Requires additional hardware.

- **Example:** Hard disk data transfer.

4. Memory-Mapped I/O (MMIO)

- I/O devices are assigned specific memory addresses.
- The CPU communicates using standard memory instructions.
- **Pros:** Simplifies programming, integrates well with modern architectures.
- **Cons:** Uses up memory address space.
- **Example:** Embedded systems and GPUs.

5. Isolated I/O (Port-Mapped I/O - PMIO)

- Uses a separate address space distinct from main memory.
- Special CPU instructions (like IN and OUT in x86) are used.
- **Pros:** Does not consume memory space.
- **Cons:** Requires special instructions.
- **Example:** Serial and parallel ports.

Comparison Table of I/O Transfer Methods

Feature	Programmed I/O (Polling)	Interrupt-Driven I/O	Direct Memory Access (DMA)	Memory-Mapped I/O (MMIO)	Isolated I/O (PMIO)
CPU Involvement	High (CPU checks constantly)	Low (CPU only handles interrupts)	Minimal (DMA controller handles transfer)	Moderate (CPU communicates using memory instructions)	Moderate (CPU uses special instructions)
Efficiency	Low (CPU stays busy)	Medium (CPU is free until interrupt)	High (CPU can do other tasks)	Medium (direct memory interaction)	Medium (uses separate I/O space)
Speed	Slow	Faster than polling	Fastest	Moderate	Moderate
Hardware Complexity	Low	Medium	High (Requires	Medium	Medium

Feature	Programmed I/O (Polling)	Interrupt-Driven I/O	Direct Memory Access (DMA)	Memory-Mapped I/O (MMIO)	Isolated I/O (PMIO)
			DMA controller)		
CPU Workload	High (wastes cycles)	Medium	Low	Medium	Medium
Example Devices	Keyboard, Mouse	Hard Disk, Printer	Graphics Card, SSD	Embedded Systems, GPUs	Serial, Parallel Ports